

Web Development Fundamentals

Assignment 1 — Introduction to Web Architecture

Name	Ansh Shukla
Roll No.	2415000257
Subject	Web Development
Assignment	Frontend, Backend & Web Architecture Basics

Q1. Explain the difference between frontend, backend, and full-stack development with suitable real-world examples.

Modern web applications are built in layers, and the three terms frontend, backend, and full-stack describe which layer a developer specializes in. Understanding this split is important because it explains how large development teams divide their work, how job roles are structured in the software industry, and how a single web page such as an Instagram feed or an Amazon product page actually comes together behind the scenes.

Frontend Development

Frontend development refers to everything a user directly sees and interacts with in their browser or app — the layout, colours, fonts, buttons, animations, and overall user experience. It is often called "client-side" development because it runs on the user's device rather than on a remote server. A frontend developer is responsible for translating a visual design (often created in tools like Figma) into working code that renders correctly across different screen sizes and browsers.

- **Core technologies:** HTML (structure), CSS (styling), and JavaScript (interactivity).
- **Popular frameworks/libraries:** React, Angular, Vue.js — these help build reusable, component-based interfaces instead of writing raw HTML/CSS/JS by hand.
- **Key responsibilities:** Responsive design, accessibility, cross-browser compatibility, state management, and performance of what loads in the browser.

Real-world example: On Instagram, the frontend is responsible for displaying the photo feed, animating the heart icon and changing its colour when a post is liked, showing the loading skeleton while images fetch, and rendering the comment box. None of this involves storing data permanently — it is purely about presentation and user interaction.

Backend Development

Backend development refers to the server-side part of an application that the user never sees directly. It handles the business logic, processes requests coming from the frontend, communicates with databases, manages authentication and security, and returns the appropriate data or confirmation back to the client. Because it "runs on the server," it is also called "server-side" development.

- **Core technologies:** Node.js, Python (Django/Flask), Java (Spring Boot), PHP, Ruby on Rails.
- **Databases used:** MySQL, PostgreSQL, MongoDB — for storing and retrieving structured or unstructured data.
- **Key responsibilities:** Writing APIs, validating and processing data, handling user authentication, managing servers, and ensuring the application scales under load.

Real-world example: When a user taps the like button on Instagram, the frontend sends a request to the backend. The backend verifies the user is logged in, increments the like count in the database, and sends a confirmation back so the frontend can update the counter on

screen. This entire process happens in a fraction of a second and is invisible to the user.

Full-Stack Development

A full-stack developer is comfortable working across both the frontend and the backend, and understands how the two layers communicate through APIs. Full-stack developers are valuable in small teams and start-ups because they can independently build a complete feature — from the visible interface down to the database — without depending on a separate specialist for each half.

Real-world example: Consider an e-commerce website. A full-stack developer might design the product listing page and "Add to Cart" button (frontend), while also writing the server logic that checks stock availability, calculates the total price, and saves the cart to the database (backend) — all as part of the same feature.

Comparison at a Glance

Aspect	Frontend	Backend	Full-Stack
Runs on	User's browser/device	Remote server	Both
Visible to user?	Yes	No	Partially
Typical languages	HTML, CSS, JS	Python, Node.js, Java	Combination of both
Focus	User experience	Data & logic	End-to-end feature delivery

Q2. Create a simple diagram showing how the client-server model works in web architecture.

The client-server model is the foundation of how the web operates. In this model, the **client** (a web browser or mobile app) initiates a request, and the **server** (a remote machine running server software) listens for that request, processes it, communicates with a database if necessary, and sends back a response.

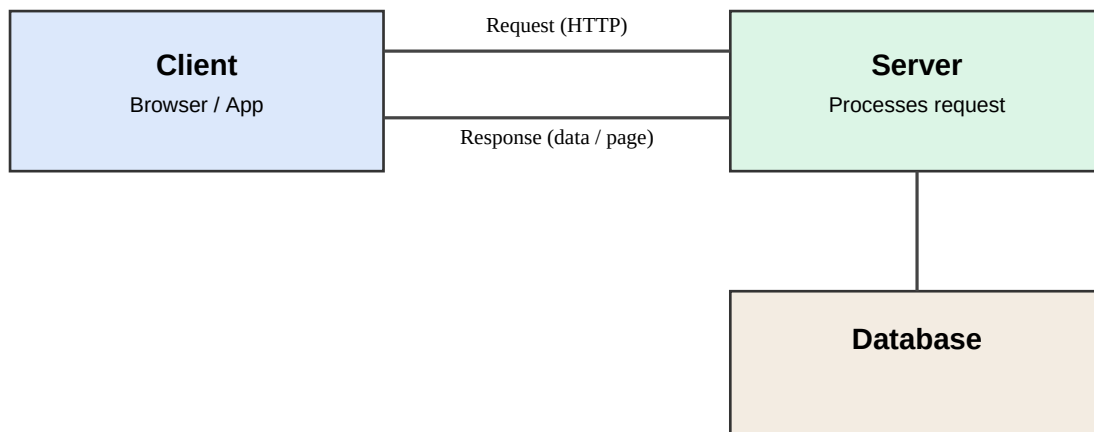


Fig. 1 — Client-Server communication model

Step-by-step explanation of the flow shown above:

- **Client initiates a request:** The user performs an action in the browser or app — typing a URL, clicking a button, or submitting a form. This is packaged as an HTTP or HTTPS request and sent over the internet to the server.
- **Server receives and processes the request:** The server software (e.g., Apache, Nginx, or a Node.js application) receives the request, determines what is being asked for, and runs the relevant backend logic.
- **Server talks to the database (if needed):** If the request requires stored data — for example, fetching a user's profile — the server queries the database, which returns the relevant records.
- **Server sends a response:** The server packages the result (HTML page, JSON data, an image, or an error message) and sends it back to the client over the same connection.
- **Client renders the response:** The browser receives the response and updates what is displayed on screen, without the user needing to do anything further.

This request-response cycle repeats every time the user interacts with the application, whether that means loading a new page, submitting a form, or fetching new data through an API call in the background (as with modern single-page applications).

Q3. Describe how a browser requests and displays a web page from a web server.

Loading a web page looks instantaneous to the end user, but under the hood it involves a precise multi-step sequence of network operations. The five stages below describe what happens between the moment a URL is entered and the moment the page appears on screen.

1. Enter URL

The user types a web address (e.g., `www.example.com`) into the browser's address bar and presses Enter, or clicks a link that points to that address.

2. DNS Lookup

Domain names are human-readable, but computers communicate using numeric IP addresses. The browser contacts a Domain Name System (DNS) server, which acts like a phonebook, translating the domain name into the correct IP address of the web server hosting that site. This result is often cached locally to speed up future visits.

3. HTTP/HTTPS Request

Once the browser has the IP address, it establishes a connection to the server (using TCP, and TLS for HTTPS) and sends an HTTP request asking for the specific resource — usually the homepage or a particular file.

4. Server Response

The web server receives the request, executes any necessary backend logic (querying a database, checking authentication, etc.), and sends back a response. This response includes a status code (e.g., 200 OK, 404 Not Found) along with the requested HTML document, and references to associated CSS, JavaScript, and image files.

5. Page Rendering

The browser's rendering engine parses the HTML to build the Document Object Model (DOM), applies CSS rules to style it, and executes JavaScript for interactivity. Images and other assets are requested and loaded as they are encountered. The final result is painted to the screen as the fully visible webpage.

Additional note: Modern browsers often perform several of these steps in parallel — for example, downloading CSS and JavaScript files simultaneously — and may also use caching, content delivery networks (CDNs), and pre-fetching techniques to make page loads noticeably faster on repeat visits.

Q4. Identify and list the tools required to set up a web development environment. Explain the purpose of each.

Setting up a productive web development environment involves more than just a text editor. Below is a categorised list of the essential tools, along with an explanation of why each one is needed in a typical development workflow.

Code Editor / IDE (e.g., VS Code)

Used to write, edit, and organise source code. Offers syntax highlighting, IntelliSense autocompletion, integrated terminal access, debugging support, and a huge extension ecosystem that speeds up everyday coding tasks.

Web Browser (e.g., Chrome, Firefox)

Used to run and visually test the website as an end user would see it. Built-in Developer Tools allow inspection of HTML/CSS/JS, live editing of styles, monitoring of network requests, and debugging of JavaScript errors.

Version Control System (e.g., Git)

Tracks every change made to the codebase over time, allows developers to revert to previous working versions if something breaks, and enables multiple developers to collaborate on the same project without overwriting each other's work through branching and merging.

Hosting Platform for Repositories (e.g., GitHub, GitLab)

Stores the project's code online so it is backed up and accessible to the whole team. Supports pull requests, code reviews, issue tracking, and often integrates directly with automated deployment pipelines (CI/CD).

Runtime Environment (e.g., Node.js)

Allows JavaScript code to run outside of the browser. This is essential for backend development with JavaScript, and also powers the build tools and package managers used in almost every modern frontend project.

Package Manager (e.g., npm, yarn)

Automates the installation, updating, and management of third-party libraries and dependencies that a project relies on, along with keeping track of exact version numbers via a lock file so the project behaves consistently on every machine.

Local Web Server (e.g., XAMPP, Live Server extension)

Simulates a real production server on the developer's own machine. This allows a website to be tested exactly as it would behave once deployed, including features like PHP processing or auto-refreshing the browser whenever a file is saved.

Database System (e.g., MySQL, MongoDB)

Stores and manages the application's persistent data, such as user accounts, product listings, or order history, and allows this data to be efficiently queried, updated, and secured.

API Testing Tool (e.g., Postman)

Used to send requests directly to backend APIs and inspect their responses, without needing a finished frontend. This is invaluable for verifying backend logic early and for documenting how an API should be used.

Browser Developer Tools

Built directly into every modern browser, these tools let a developer inspect individual HTML elements, view console errors and warnings in real time, analyse network requests and their timing, and profile performance bottlenecks.

Together, these tools cover the full lifecycle of a web project: writing code, testing it locally, collaborating with a team, and preparing it for real-world deployment.

Q5. Explain what a web server is and give examples of commonly used servers.

A web server is software (typically running continuously on a dedicated machine) that stores, processes, and delivers web content to clients whenever they request it. It listens for incoming connections — usually over the HTTP or HTTPS protocol — interprets each request, and sends back an appropriate response, which could be an HTML page, an image, a JSON payload, or an error message.

A useful analogy is to think of a web server as a waiter in a restaurant: the client places an order (a request), the waiter relays it to the kitchen (the application logic and database), and then brings back exactly what was ordered (the response).

How a Web Server Works

- The client (browser) sends a request — for example, opening a URL or submitting a form.
- The web server receives this request on a specific port (commonly port 80 for HTTP or 443 for HTTPS) and interprets what resource or action is being requested.
- If dynamic content is required, the server may pass the request to an application layer, which might query a database or run business logic.
- The server assembles the appropriate response and sends it back to the client, along with an HTTP status code indicating success or failure.

Commonly Used Web Servers

- **Apache HTTP Server** — One of the oldest and most widely used web servers, open-source and highly configurable through modules, making it flexible for a huge range of hosting setups.
- **Nginx** — Known for its high performance and ability to efficiently handle a very large number of concurrent connections; frequently used not just as a web server but also as a reverse proxy and load balancer in front of other application servers.
- **Microsoft IIS (Internet Information Services)** — A web server built into Windows Server, commonly used to host ASP.NET applications in enterprise environments.
- **LiteSpeed** — A commercial, high-performance web server designed as a drop-in replacement for Apache, popular for its speed on shared hosting platforms.
- **Node.js (via frameworks like Express)** — While technically a JavaScript runtime rather than a dedicated web server program, it is extremely common for building lightweight, custom web servers directly in JavaScript.

Q6. Define the roles of a frontend developer, backend developer, and database administrator in a project.

In any reasonably-sized software project, responsibilities are divided among specialists so that each part of the system receives focused expertise. Below are the three core roles commonly found in a web development team.

Frontend Developer

- Designs and builds the user interface (UI) that end users interact with directly.
- Ensures the website or application is responsive, visually consistent, and easy to use across different devices and screen sizes.
- Works primarily with HTML, CSS, and JavaScript, often using frameworks such as React, Angular, or Vue to build reusable UI components.
- Collaborates closely with UI/UX designers to translate visual mock-ups into functioning code, and with backend developers to correctly consume APIs.

Example: Building the product listing page layout, the image carousel, and the styling and click behaviour of the "Add to Cart" button on an e-commerce site.

Backend Developer

- Builds and maintains the server-side logic that powers the application's core functionality.
- Handles data processing, business rules, authentication, authorisation, and security.
- Designs and implements APIs that the frontend (or other services) can call to fetch or modify data.
- Works with languages such as Node.js, Python, or Java, and connects the application to its database layer.

Example: Writing the logic that checks whether an item is in stock the moment a user clicks "Add to Cart," and rejecting the request with an appropriate error message if it is out of stock.

Database Administrator (DBA)

- Manages, organises, and secures the databases in which an application's data is stored.
- Ensures data integrity through proper schema design, constraints, and validation rules.
- Performs regular backups and disaster-recovery planning to prevent data loss.
- Optimises slow queries and indexes to keep the application performing well as data volume grows.
- Controls access permissions so that only authorised users or services can read or modify sensitive data.

Example: Structuring the product and order tables so that lookups remain fast even as the catalogue grows to millions of items, and making sure a server crash never results in lost order records.

Q7. Install VS Code and configure it for HTML, CSS, and JavaScript development. Take a screenshot of the setup.

The following steps document the complete process of installing and configuring Visual Studio Code as a working environment for HTML, CSS, and JavaScript development.

Download VS Code

Visited the official website code.visualstudio.com and downloaded the installer suitable for the operating system in use (Windows, macOS, or Linux).

Installation

Ran the downloaded installer, accepted the license agreement, kept the default installation options (including adding VS Code to PATH for easy terminal access), and completed the setup. VS Code launched successfully immediately after installation finished.

Installing Extensions for Web Development

Opened the Extensions panel using `Ctrl+Shift+X` and installed a set of extensions tailored to frontend development:

- **Live Server** — Runs the HTML file on a local development server and automatically refreshes the browser every time a file is saved, removing the need to manually reload.
- **Prettier** — Automatically formats HTML, CSS, and JavaScript code according to consistent style rules, saving time on manual formatting and reducing style-related disagreements in team projects.
- **HTML CSS Support** — Provides autocomplete suggestions for CSS class and ID names directly inside HTML files, reducing typing errors.
- **Auto Rename Tag** — Automatically renames the matching closing HTML tag whenever the opening tag is edited, preventing mismatched tags.

Creating a Project Folder

Created a new folder for the project and opened it in VS Code via `File → Open Folder`. Inside this folder, three files were created: `index.html`, `style.css`, and `script.js`, following the standard structure used in almost every frontend project.

Linking the Files Together

The CSS file was linked inside the `<head>` of `index.html` using a `<link>` tag, and the JavaScript file was linked near the end of the `<body>` using a `<script>` tag, so that all three files work together as a single functioning page.

Testing the Setup

Right-clicked on `index.html` and selected "Open with Live Server." This launched the page in the default browser on a local address (typically `http://127.0.0.1:5500`), and any change made and saved inside VS Code was reflected instantly in the browser without a manual refresh —

confirming that the development environment was configured correctly and ready for use.

Q8. Explain the difference between static and dynamic websites. Provide an example of each.

Static Websites

A static website consists of fixed content that is sent to the browser exactly as it is stored on the server, with no processing required to generate the page. Every visitor sees the same HTML, CSS, and images unless a developer manually edits the source files. Because there is no server-side computation involved, static sites tend to load very quickly and are simple and inexpensive to host.

- Built using only HTML and CSS (JavaScript may still be used for front-end interactivity, but the content itself does not change per request).
- No database or backend server logic is required.
- Easy to deploy on simple hosting services or content delivery networks.

Example: A personal portfolio website showcasing a resume, project links, and contact details — the same for every visitor.

Dynamic Websites

A dynamic website generates content on the fly, often customised based on user input, account data, or real-time information pulled from a database. Each time the page is requested, the server may run backend code to assemble a response that can differ from one visitor to the next, or even from one moment to the next for the same visitor.

- Built using HTML, CSS, and JavaScript on the frontend, combined with a backend language and a database.
- Content changes based on factors such as logged-in user, search queries, or live inventory.
- Requires more infrastructure — a running server process and typically a database connection.

Example: An e-commerce platform such as Amazon, where product prices, stock levels, personalised recommendations, and order history are different for every user and change constantly.

Side-by-Side Comparison

Aspect	Static Website	Dynamic Website
Content	Fixed for every visitor	Changes based on data/user
Technology	HTML, CSS	HTML, CSS, JS + backend + database
Speed	Generally faster (no processing)	Slightly slower (server-side processing involved)
Hosting complexity	Simple, cheap	Requires server + database infrastructure
Example	Portfolio site	E-commerce site (Amazon)

Q9. Research and list five web browsers. Explain how rendering engines differ between them.

A rendering engine is the part of a browser responsible for taking raw HTML, CSS, and JavaScript and converting it into the visual webpage that appears on screen. Although all modern engines aim to follow the same web standards, small differences in implementation can cause a page to look or behave slightly differently depending on which browser is used — which is why cross-browser testing remains an important part of web development.

1. Blink (Chrome, Edge, Opera, Brave)

- Developed by Google, forked from WebKit in 2013.
- Delivers fast performance and strong, rapid support for the latest web standards and JavaScript features.
- Since the majority of browsers today are Blink-based, its behaviour has effectively become the default that most developers test against first.

2. Gecko (Firefox)

- Developed by Mozilla as an open-source project.
- Known for strict adherence to official web standards and a strong focus on privacy and security features.
- Occasionally renders CSS spacing, font rendering, or animation timing slightly differently compared to Blink, requiring developers to double-check layouts in Firefox.

3. WebKit (Safari)

- Developed originally by Apple, and historically the common ancestor that Blink itself was forked from.
- Highly optimised for Apple's own hardware, offering better battery efficiency and smooth performance on macOS and iOS devices.
- Tends to be slower to adopt certain brand-new web standards compared to Blink, which can occasionally cause compatibility issues for developers relying on cutting-edge features.

4. EdgeHTML (Legacy Microsoft Edge)

- Microsoft's original in-house engine used before Edge switched over to Blink in 2020.
- Built for tight integration with Windows, but eventually discontinued in favour of Blink to gain better compatibility with the wider web.

5. Presto (Legacy Opera)

- Used by Opera prior to its move to the Blink engine.
- Known at the time for fast page loading and compact resource usage, though it is no longer maintained or used by any current browser.

Why this matters for developers: Because rendering engines can interpret certain CSS properties, animation timings, or JavaScript APIs slightly differently, thorough web development practice includes testing a site across at least a Blink-based browser (e.g., Chrome), Gecko (Firefox), and WebKit (Safari) to catch any inconsistencies before deployment.

Q10. Draw a labeled diagram showing the basic web architecture flow — client, server, database, and APIs.

Modern web applications typically add an API layer between the client and the server. The API defines a clear contract for how the client and server are allowed to communicate — what data can be requested, in what format, and under what conditions — which keeps the frontend and backend loosely coupled and independently maintainable.

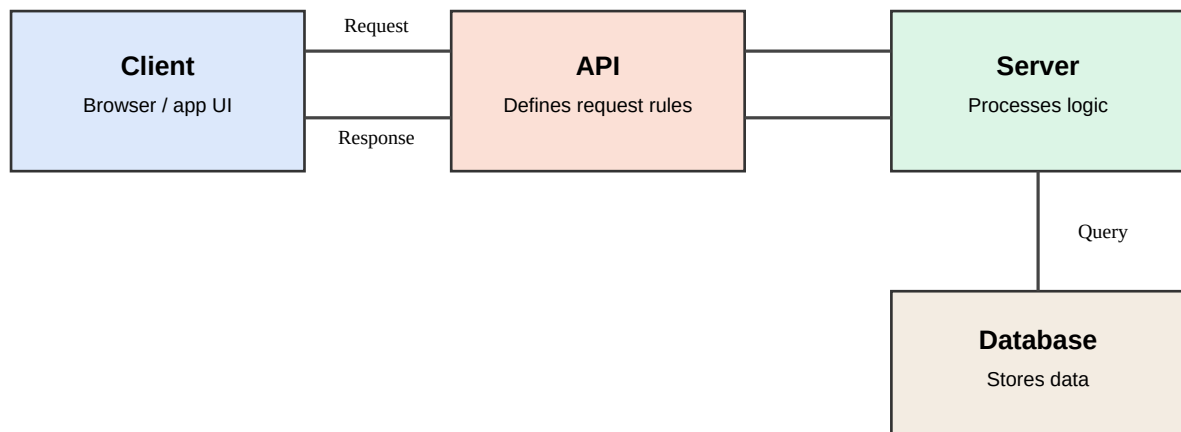


Fig. 2 — Client → API → Server → Database architecture flow

Component Roles

- **Client:** The user-facing part of the system — a browser or mobile app — where all requests originate based on user actions.
- **API (Application Programming Interface):** Acts as a well-defined middleman between client and server, specifying exactly what data can be requested, what format it will be returned in (commonly JSON), and what operations are permitted.
- **Server:** Receives the request forwarded through the API, executes the relevant business logic, applies validation and security checks, and interacts with the database whenever persistent data is required.
- **Database:** Stores and retrieves the actual data used by the application — such as user accounts, product catalogues, or order history — and returns query results back to the server.

Example Flow: Viewing a User Profile

1. The client clicks "View Profile" in the app.
2. This action is translated into a request that is sent through the API according to its defined rules (e.g., a GET request to `/api/users/123`).
3. The server receives this request via the API and queries the database for the corresponding user record.
4. The database returns the matching data back to the server.

5. The server formats this data and sends it back through the API as a response.
6. The client receives the response and renders the profile information on screen for the user to see.

This layered flow — client, API, server, database — is the backbone of virtually every modern web and mobile application, from simple personal projects to large-scale platforms serving millions of users.